# DL1

**Linear regression by using Deep Neural network:** Implement Boston housing price prediction problem by Linear regression using Deep Neural network. Use Boston House price prediction dataset

```
# ==============================
# Step 1: Import Libraries
# ==============================

import numpy as np          # For numerical operations (arrays, math)
import pandas as pd         # For handling dataset (DataFrame)
import matplotlib.pyplot as plt   # For plotting graphs

from sklearn.model_selection import train_test_split   # To split data into train & test
from sklearn.preprocessing import StandardScaler       # To normalize data

import tensorflow as tf
from tensorflow.keras.models import Sequential         # For building neural network
from tensorflow.keras.layers import Dense, Input       # Dense = fully connected layer
from tensorflow.keras.callbacks import EarlyStopping   # To stop training early (avoid overfitting)
```

```
# ==============================
# Step 2: Load Dataset
# ==============================

df = pd.read_csv("1_boston_housing.csv")
# Load Boston housing dataset from CSV file
```

```
# Remove unwanted quotes from column names (clean column names)
df.columns = df.columns.str.replace('"', '')

# Display first 5 rows of dataset
print(df.head())



# ==============================
# Step 3: Split Features & Target
# ==============================

X = df.drop("MEDV", axis=1)
# X = input features (all columns except MEDV)

y = df["MEDV"]
# y = target variable (house price)

print("Shape of X:", X.shape)   # Number of rows & features
print("Shape of y:", y.shape)   # Number of target values



# ==============================
# Step 4: Train-Test Split
# ==============================

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

# 80% data → training
# 20% data → testing
# random_state ensures same split every time



# ==============================
# Step 5: Feature Scaling
# ==============================

scaler = StandardScaler()
# Standardize data (mean = 0, std = 1)

X_train = scaler.fit_transform(X_train)
# Fit scaler on training data and transform it

X_test = scaler.transform(X_test)
# Only transform test data (no fitting)



# ==============================
# Step 6: Build Deep Neural Network
# ==============================

model = Sequential([
```

```
    Input(shape=(X_train.shape[1],)),
    # Input layer with number of features

    Dense(64, activation='relu'),
    # Hidden layer 1 (64 neurons, ReLU activation)

    Dense(32, activation='relu'),
    # Hidden layer 2 (32 neurons)

    Dense(16, activation='relu'),
    # Hidden layer 3 (16 neurons)

    Dense(1, activation='linear')
    # Output layer → 1 value (house price)
])

# Compile the model
model.compile(
    optimizer='adam',     # Optimizer (adjusts weights)
    loss='mse',           # Loss function (Mean Squared Error)
    metrics=['mae']       # Metric (Mean Absolute Error)
)

# Display model architecture
model.summary()



# ==============================
```

```
# Step 7: Train Model
# ==============================

early_stop = EarlyStopping(
    monitor='val_loss',        # Monitor validation loss
    patience=10,               # Stop if no improvement for 10 epochs
    restore_best_weights=True  # Restore best model weights
)

history = model.fit(
    X_train, y_train,
    epochs=200,                # Maximum number of iterations
    batch_size=16,             # Number of samples per batch
    validation_split=0.2,      # 20% of training used for validation
    callbacks=[early_stop],    # Apply early stopping
    verbose=1                  # Show training progress
)



# ==============================
# Step 8: Evaluate Model
# ==============================

loss, mae = model.evaluate(X_test, y_test)
# Evaluate model on unseen test data

print("\nTest Loss (MSE):", loss)
print("Test MAE:", mae)
```

```
# ==============================
# Step 9: Predictions
# ==============================

y_pred = model.predict(X_test)
# Predict house prices for test data

print("\nActual vs Predicted (First 5 values):")
for i in range(5):
    print(f"Actual: {y_test.iloc[i]:.2f} | Predicted: {y_pred[i][0]:.2f}")
    # Compare real vs predicted values



# ==============================
# Step 10: Plot Training Graphs
# ==============================

# Plot Loss Graph
plt.figure()
plt.plot(history.history['loss'], label='Training Loss')       # Training loss
plt.plot(history.history['val_loss'], label='Validation Loss') # Validation loss
plt.legend()
plt.title("Loss vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
```

```
plt.show()

# Plot MAE Graph
plt.figure()
plt.plot(history.history['mae'], label='Training MAE')         # Training MAE
plt.plot(history.history['val_mae'], label='Validation MAE')   # Validation MAE
plt.legend()
plt.title("MAE vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("MAE")
plt.show()
```

## 🎯 START YOUR EXPLANATION LIKE THIS

👉
"Sir, this program implements **house price prediction using a Deep Neural Network**, which is an advanced form of linear regression using multiple layers."

---

## ✅ LINE-BY-LINE EXPLANATION (VERY SIMPLE LANGUAGE)

---

### 🔹 Step 1: Import Libraries

👉 Say this:

- numpy → used for numerical operations
- pandas → used to handle dataset
- matplotlib → used for graphs
- train_test_split → used to divide data into training and testing
- StandardScaler → used to normalize data
- tensorflow/keras → used to build neural network

👉 One-line:

"These libraries are used for data handling, preprocessing, and building the deep learning model."

---

### 🔹 Step 2: Load Dataset

df = pd.read_csv("1_boston_housing.csv")

👉 Say:

"Here I load the Boston Housing Dataset from a CSV file."

---

df.columns = df.columns.str.replace("", ")

👉 Say:

"This line removes unwanted quotes from column names to clean the dataset."

---

print(df.head())

👉 Say:

“This displays the first 5 rows to understand the dataset.”

---

🔹 **Step 3: Split Features & Target**

```
X = df.drop("MEDV", axis=1)
y = df["MEDV"]
```

👉 Say:

“Here I separate input features and output.
All columns except MEDV are inputs, and MEDV is the target (house price).”

---

🔹 **Step 4: Train-Test Split**

```
train_test_split(...)
```

👉 Say:

“I divide the dataset into 80% training and 20% testing data so that the model can be evaluated on unseen data.”

---

🔹 **Step 5: Feature Scaling**

```
scaler = StandardScaler()
```

👉 Say:

“I use StandardScaler to normalize the data.”

---

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

👉 Say:

“Training data is fitted and transformed, while test data is only transformed using the same scaler.”

👉 Important line:

“Scaling improves model performance and convergence.”

---

🔹 **Step 6: Build Neural Network**

model = Sequential([

👉 Say:

"Here I create a sequential deep neural network model."

---

Input(shape=(X_train.shape[1],))

👉 Say:

"Input layer takes number of features as input."

---

Dense(64, activation='relu')

👉 Say:

"First hidden layer with 64 neurons and ReLU activation."

---

Dense(32, activation='relu')
Dense(16, activation='relu')

👉 Say:

"Additional hidden layers to learn complex patterns."

---

Dense(1, activation='linear')

👉 Say:

"Output layer gives a single value, which is the predicted house price."

👉 Important:

"Linear activation is used because this is a regression problem."

---

🔹 **CORE IDEA (Say confidently)**

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

👉 Say:

"Internally, the model learns this linear relationship, but hidden layers improve prediction accuracy."

---

🔹 **Compile Model**

model.compile(optimizer='adam', loss='mse', metrics=['mae'])

👉 Say:

"Adam optimizer is used to update weights, MSE is used as loss function, and MAE is used as evaluation metric."

---

🔹 **Step 7: Train Model**

EarlyStopping(...)

👉 Say:

"Early stopping is used to prevent overfitting."

---

model.fit(...)

👉 Say:

"The model is trained on training data for multiple epochs."

👉 Add:

"Validation data is used to monitor performance."

---

🔹 **Step 8: Evaluate Model**

model.evaluate(...)

👉 Say:

"Here I evaluate model performance on test data using loss and MAE."

---

🔹 **Step 9: Predictions**

model.predict(...)

👉 Say:

"The model predicts house prices."

---

print(actual vs predicted)

👉 Say:

“I compare actual and predicted values to check accuracy.”

---

🔹 **Step 10: Graphs**

plt.plot(...)

👉 Say:

“These graphs show how loss and MAE decrease during training.”

👉 Important:

“If validation loss increases, it indicates overfitting.”

---

🔥 **FINAL 20-SECOND SUMMARY (VERY IMPORTANT)**

👉 Say this at end:

“In this project, I implemented linear regression using a deep neural network on the Boston dataset.
The model learns from training data, evaluates on test data, and predicts house prices with good accuracy.”

## 🎯 WHAT IS DRY RUN?

👉 It means:

"Explaining how data flows through the program using sample values."

---

## ✅ STEP-BY-STEP DRY RUN (WITH VALUES)

---

### 🔹 Step 1: Load Dataset

Assume first row of Boston Housing Dataset:

CRIM = 0.00632
ZN = 18.0
INDUS = 2.31
RM = 6.575
LSTAT = 4.98
MEDV = 24.0

👉 Say:

"Each row represents one house with multiple features and target price MEDV."

---

### 🔹 Step 2: Split Features & Target

```
X = df.drop("MEDV", axis=1)
y = df["MEDV"]
```

👉 Now:

X = [0.00632, 18.0, 2.31, 6.575, 4.98, ...]
y = 24.0

👉 Say:

"X contains input features, and y contains output price."

---

### 🔹 Step 3: Train-Test Split

Assume:

- Training data → 400 rows
- Testing data → 100 rows

👉 Say:

"Model learns from training data and is tested on unseen data."

---

🔹 **Step 4: Feature Scaling**

Before scaling:

RM = 6.575
LSTAT = 4.98

After scaling:

RM = 0.52
LSTAT = -0.30

👉 Say:

"All values are converted to similar scale for better learning."

---

🔹 **Step 5: Input to Neural Network**

Example input:

X_train[0] = [0.12, -0.45, 0.67, 0.52, -0.30, ...]

👉 Say:

"This scaled feature vector is given as input to the neural network."

---

🔹 **Step 6: Forward Pass (Layer by Layer)**

---

🔸 **Layer 1 (64 neurons)**

Each neuron calculates:

$$z = w_1 x_1 + w_2 x_2 + \cdots + b$$

Assume output:

[0.8, 0.2, 0, 1.5, ..., 64 values]

👉 Say:

"Each neuron learns a combination of input features."

---

🔸 **Layer 2 (32 neurons)**

Input → previous layer output

Output:

[1.2, 0.5, 0, ..., 32 values]

---

🔸 **Layer 3 (16 neurons)**

Output:

[0.9, 0.3, ..., 16 values]

---

🔸 **Output Layer**

Final prediction:

Predicted Price = 23.5

👉 Say:

"Model predicts house price based on learned patterns."

---

🔹 **Step 7: Loss Calculation**

Actual price:

y = 24.0
prediction = 23.5

Error:

(24 - 23.5)^2 = 0.25

👉 Say:

"Model calculates error using Mean Squared Error."

---

🔹 **Step 8: Backpropagation**

👉 Say:

"The model adjusts weights to reduce error using optimizer."

---

### 🔹 Step 9: Training Over Epochs

Epoch 1:

Loss = 100

Epoch 50:

Loss = 30

Epoch 100:

Loss = 15

👉 Say:

"Loss decreases as model learns."

---

### 🔹 Step 10: Testing Phase

Input (test data):

[0.10, -0.20, 0.60, ...]

Output:

Predicted = 21.8
Actual = 22.6

👉 Say:

"Model predicts close to actual value."

---

### 🔹 Step 11: Output Comparison

Actual: 24.0 | Predicted: 23.5
Actual: 22.6 | Predicted: 21.8

👉 Say:

"Predicted values are close to actual values, so model is performing well."

---

### 🔹 Step 12: Graphs

Loss graph:

- Decreasing → good learning
- Increasing → overfitting

👉 Say:

“Graphs show how model improves over time.”

---

## 🔥 FINAL DRY RUN SUMMARY (SAY THIS)

👉

“Input features are scaled and passed through multiple neural network layers.
Each layer extracts patterns, and finally, the model predicts house price.
Error is calculated and minimized using backpropagation over multiple epochs.”

---

## 💥 PRO LEVEL LINE (USE THIS)

👉

“This model performs regression using deep learning by approximating a linear function with multiple non-linear transformations.

📊 **GRAPH 1: Loss vs Epochs**

🎯 **What this graph shows**

- **X-axis → Epochs (training iterations)**
- **Y-axis → Loss (MSE error)**

---

🧠 **How to explain (say this)**

👉

**"This graph shows how the model error decreases during training."**

---

🔍 **Observation**

**1. Initial Stage (Epoch 0–5)**

- **Loss is very high (~600)**
- **Then sharp drop**

👉 **Say:**

**"Initially model does not know anything, so error is high. After learning starts, error decreases rapidly."**

---

**2. Middle Stage (Epoch 5–20)**

- **Loss reduces gradually**

👉 **Say:**

**"Model is learning patterns from data."**

---

**3. Final Stage (After ~20 epochs)**

- **Loss becomes stable (~10–15)**

👉 **Say:**

**"Model has converged, meaning learning is almost complete."**

---

🔵 **Training vs** 🟠 **Validation**

👉 **Important point:**

- **Both lines are very close**
- **No large gap**

👉 **Say:**

**“Since training and validation loss are close, the model is not overfitting.”**

---

⚠️ **Examiner Trap Question**

👉 **If asked:**

**“What if validation loss increases?”**

**Answer:**

**“That indicates overfitting.”**

📊 **GRAPH 2: MAE vs Epochs**

🎯 **What this graph shows**

- **X-axis → Epochs**
- **Y-axis → MAE (Mean Absolute Error)**

---

🧠 **How to explain**

👉

**“This graph shows how prediction error reduces during training.”**

---

🔍 **Observation**

**1. Initial Stage**

- **MAE is high (~22)**

👉 **Say:**

**"Initial predictions are far from actual values."**

---

**2. Learning Phase**

- **MAE quickly drops to ~5**

👉 **Say:**

**"Model starts predicting closer values."**

---

**3. Final Stage**

- **MAE stabilizes around 2–3**

👉 **Say:**

**"Final prediction error is very small, indicating good accuracy."**

---

🔵 **Training vs** 🟠 **Validation**

👉 **Observation:**

- **Lines are close**
- **Validation slightly higher**

👉 **Say:**

**"Validation error is slightly higher, which is expected, but still close → good generalization."**

---

🔥 **FINAL 10-SECOND SUMMARY (VERY IMPORTANT)**

👉 **Say this:**

**"Both graphs show decreasing error and stable convergence. Training and validation curves are close, which means the model is well-trained and not overfitting."**

---

💥 **PRO LEVEL LINE (Use this to impress)**

👉

**“The model achieves good bias-variance balance, as both training and validation errors converge without divergence.”**

## 🎯 MOST IMPORTANT VIVA QUESTIONS (WITH PERFECT ANSWERS)

---

### 🔹 1. What is the objective of your project?

👉

**"The objective is to predict house prices using a Deep Neural Network, which performs regression on the Boston Housing Dataset."**

---

### 🔹 2. What type of problem is this?

👉

**"This is a regression problem because the output is a continuous value (house price)."**

---

### 🔹 3. What is linear regression?

👉

**"Linear regression predicts output using a linear equation of input features."**

---

### 🔹 4. Why use Deep Neural Network instead of linear regression?

👉

**"DNN can capture complex and non-linear relationships between features, giving better accuracy."**

---

### 🔹 5. Why is the output layer linear?

👉

**"Because regression problems require continuous output values."**

---

### 🔹 6. What is the role of hidden layers?

👉

**"Hidden layers learn patterns and relationships between input features."**

---

🔹 **7. Why did you use ReLU activation?**

👉

**“ReLU helps avoid vanishing gradient and improves training speed.”**

---

🔹 **8. What is MSE?**

👉

**“Mean Squared Error measures the average squared difference between actual and predicted values.”**

---

🔹 **9. What is MAE?**

👉

**“Mean Absolute Error is the average absolute difference between predicted and actual values.”**

---

🔹 **10. Difference between MSE and MAE?**

👉

**“MSE penalizes large errors more, while MAE gives equal weight to all errors.”**

---

🔹 **11. What is overfitting?**

👉

**“Overfitting occurs when the model learns training data too well but performs poorly on new data.”**

---

🔹 **12. How did you prevent overfitting?**

👉

**“I used EarlyStopping and validation data to monitor performance.”**

---

🔹 **13. What is EarlyStopping?**

👉

**“It stops training when validation loss stops improving, preventing overfitting.”**

---

🔹 **14. What is feature scaling?**

👉

**“It normalizes data so all features are on the same scale.”**

---

🔹 **15. Why is scaling important?**

👉

**“It improves model convergence and training speed.”**

---

🔹 **16. What is train-test split?**

👉

**“It divides data into training and testing sets to evaluate model performance.”**

---

🔹 **17. What is epoch?**

👉

**“One complete pass of the entire dataset through the model.”**

---

🔹 **18. What is batch size?**

👉

**“Number of samples processed before updating weights.”**

---

🔹 **19. What optimizer did you use?**

👉

**“Adam optimizer, because it adapts learning rate and converges faster.”**

---

🔹 **20. What is backpropagation?**

👉

**“It updates weights by calculating gradients of loss.”**

---

🔥 **MEDIUM LEVEL QUESTIONS (VERY IMPORTANT)**

---

🔹 **21. Why multiple hidden layers?**

👉

**“To learn complex patterns and improve prediction accuracy.”**

---

🔹 **22. What happens if we don’t use scaling?**

👉

**“Training becomes slow and model may not converge properly.”**

---

🔹 **23. What is validation data?**

👉

**“It is used during training to check model performance.”**

---

🔹 **24. How do you know model is good?**

👉

**“Low loss and MAE, and close training and validation curves.”**

---

🔹 **25. What does your graph show?**

👉

**“Loss and MAE decrease over epochs, indicating successful learning.”**

---

## ⚠️ TRICKY QUESTIONS (EXAMINER FAVORITE)

---

### 🔹 26. Is this really linear regression?

👉

**“Yes, because output layer is linear, but hidden layers add non-linear transformations making it more powerful.”**

---

### 🔹 27. Why not use sigmoid in output?

👉

**“Sigmoid limits output between 0 and 1, which is not suitable for price prediction.”**

---

### 🔹 28. What if model overfits?

👉

**“Validation loss will increase while training loss decreases.”**

---

### 🔹 29. What is underfitting?

👉

**“Model fails to learn patterns, resulting in high error.”**

---

### 🔹 30. How can you improve your model?

👉

**“By tuning layers, neurons, learning rate, or adding more data.”**

---

## 💥 SUPER IMPRESS ANSWER (USE THIS)

👉 **If examiner asks “Explain your model in 1 line”**

**“This model performs regression using a deep neural network, where multiple hidden layers transform input features to improve prediction accuracy beyond traditional linear regression.”**